

Practice 14 详解答案

A. 单项选择题

1. **B**: 只能重载已有运算符，为用户自定义类型定义合理行为。
2. **C**: `<<` 是流插入运算符，常用于 `cout << object`。
3. **B**: `friend` 可授予指定的非成员函数访问私有成员的权限。
4. **A**: `friend void display(Note);` 是合法的友元函数声明。
5. **A**: `cout << note` 的左操作数是 `ostream`，不能把 `operator<<` 写成 `Note` 的普通成员函数。
6. **C**: 友元函数仍是普通非成员函数，只是获得了特殊访问权限。
7. **A**: `>>` 是流提取运算符，常用于从 `cin` 读取对象。
8. **B**: 只在确实需要访问私有数据时使用友元。
9. **C**: `operator<<` 通常返回 `ostream&`。
10. **A**: 让自定义类型以自然、一致的方式使用。

B. 填空题

1. 用户（自定义）
2. `friend`
3. `<<`
4. `>>`
5. 不是

C. 判断题

1. **错**: 只能改变已有运算符对自定义类型的行为，不能创造新符号。
2. **对**: 被声明为友元后可读取和修改私有成员。
3. **错**: 许多运算符适合写成成员函数，例如 `operator==`。
4. **对**: `operator<<` 支持 `cout << object`。
5. **错**: 友元关系是单向的，需要分别声明。
6. **错**: 友元关系不继承。
7. **错**: 运算符优先级和结合性不能通过重载改变。
8. **对**: 例如 `+` 应表达组合或相加，`==` 应表达相等比较。
9. **对**: 友元函数是普通函数，可按普通函数规则调用。
10. **对**: 过多友元会让外部代码依赖类的内部表示。

D. 代码阅读

1. 声明含义

```
friend ostream& operator<<(ostream& out, const Note& note);
```

这是一个非成员友元函数声明。它重载 <<，可以访问 Note 的私有成员；第一个参数是输出流，第二个参数是只读笔记；返回输出流的引用，从而支持：

```
cout << firstNote << secondNote;
```

2. 输出

题面 Point(intx) 应写成 Point(int x)。修正后：

```
Point p(5);  
cout << p.x;
```

输出是 5。初始化列表左侧 x 是成员，右侧 x 是构造函数参数。

3. operator<< 的问题

```
ostream operator<<(ostream out, const Note& note)  
{  
    return out;  
}
```

输出流不应按值传递或按值返回。ostream 不可复制，并且按值会丢失原输出流的状态。正确签名应为：

```
ostream& operator<<(ostream& out, const Note& note)  
{  
    // out << note 的数据成员;  
    return out;  
}
```

4. 重载的运算符

```
bool operator==(const Note& other) const;
```

重载的是相等运算符 `==`。末尾 `const` 表明比较不会修改当前对象。

5. 为什么失败

`title` 是 `private`，而 `print` 是类外普通函数：

```
cout << note.title; // 编译错误
```

解决方式是给类提供 `public` `getter`/显示成员函数，或在确有必要时将 `print` 声明为 `friend`。

E. 简答题

1. 运算符重载如何提高可用性

重载让对象使用符合其含义的 C++ 语法。例如 `cout << note` 可显示笔记，`a == b` 可比较两本书，`a + b` 可组合计数器。调用者不必记忆不自然的辅助函数名，接口也更接近内置类型；前提是含义必须直观。

2. 友元函数与封装的关系

`private` 默认隐藏类的实现，友元是类主动授予某个指定函数的受控例外。友元函数不是成员函数，但可访问私有数据。它适合输入输出等需要访问内部表示的少数操作；过度使用会破坏封装，应优先考虑成员函数或公开的最小接口。

3. 为什么 `operator<<` 通常是非成员友元

表达式 `cout << note` 等价于 `operator<<(cout, note)`。左操作数是标准库的 `ostream`，不能向 `ostream` 类添加 `Note` 的成员函数。因此 `operator<<` 必须是非成员函数；要读取 `Note` 私有成员时可声明为友元。

F. 编程题

1. 重载输出运算符

```
#include <iostream>
#include <string>
using namespace std;

class Student
{
private:
    string name;
    int age;

public:
    Student(const string& studentName, int studentAge)
        : name(studentName), age(studentAge)
    {
    }

    friend ostream& operator<<(ostream& out, const Student& student)
    {
        out << "Name: " << student.name << ", Age: " << student.age;
        return out;
    }
};

int main()
{
    Student student("Alice", 20);
    cout << student << '\n';
}
```

2. 重载相等运算符

```
#include <iostream>
#include <string>
using namespace std;

class Book
{
private:
    string title;
    string author;

public:
    Book(const string& bookTitle, const string& bookAuthor)
        : title(bookTitle), author(bookAuthor)
    {
    }

    bool operator==(const Book& other) const
    {
        return title == other.title && author == other.author;
    }
};

int main()
{
    Book a("C++ Primer", "Lippman");
    Book b("C++ Primer", "Lippman");
    cout << (a == b) << '\n';
}
```

输出为 1，因为两个比较条件都成立。

3. 重载加法运算符

```
#include <iostream>
using namespace std;

class Counter
{
private:
    int value;

public:
    Counter(int initialValue) : value(initialValue)
    {
    }

    Counter operator+(const Counter& other) const
    {
        return Counter(value + other.value);
    }

    int getValue() const
    {
        return value;
    }
};

int main()
{
    Counter a(3);
    Counter b(7);
    Counter c = a + b;
    cout << c.getValue() << '\n';
}
```

`a + b` 等价于 `a.operator+(b)`，返回一个值为 10 的新对象。

4. 友元函数练习

```
#include <iostream>
using namespace std;

class Rectangle
{
private:
    int width;
    int height;

public:
    Rectangle(int rectangleWidth, int rectangleHeight)
        : width(rectangleWidth), height(rectangleHeight)
    {
    }

    friend int area(const Rectangle& r);
};

int area(const Rectangle& r)
{
    return r.width * r.height;
}

int main()
{
    Rectangle rectangle(4, 5);
    cout << area(rectangle) << '\n';
}
```

输出 20。area 虽然不是成员函数，但因为是友元而能读取私有的宽和高。

5. CLI Notes 美观打印（挑战）

```
#include <iostream>
#include <string>
using namespace std;

class Note
{
private:
    string title;
    string content;
    string tag;

public:
    Note(const string& noteTitle, const string& noteContent, const string& noteTag)
        : title(noteTitle), content(noteContent), tag(noteTag)
    {
    }

    bool operator==(const Note& other) const
    {
        return title == other.title;
    }

    friend ostream& operator<<(ostream& out, const Note& note)
    {
        out << "===== NOTE =====\n";
        out << "Title   : " << note.title << '\n';
        out << "Content : " << note.content << '\n';
        out << "Tag     : " << note.tag << '\n';
        out << "=====";
        return out;
    }
};

int main()
{
    Note meeting("Meeting", "Discuss project", "Work");
    Note sameTitle("Meeting", "Another content", "General");

    cout << meeting << "\n\n";
    cout << (meeting == sameTitle) << '\n';
}
```

operator== 按题意只比较标题，所以示例中比较结果是 1。

G. 设计与思考挑战

`cout << note` 更符合 C++ 对可打印类型的惯例，特别适合日志、调试和连续输出，例如 `cout << "Saved: " << note;`。它需要重载 `operator<<`，并且输出格式应是清晰、稳定的。`note.display()` 也完全合理，尤其当显示是一个有明确副作用的业务操作；二者可同时提供。对于“把对象写入输出流”这一用途，设计 B 更惯用。